智元D1开发教程 | Sim2Real全指南（下）——从训练诊断到 Jetson Orin 实机部署

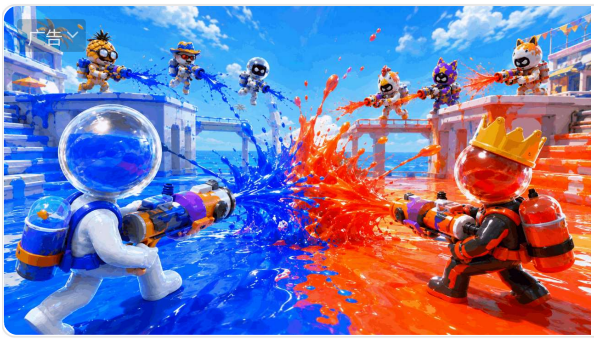
原创 敢敢不敢写bug 敢敢AUTOHUB 2026年4月21日 08:45 上海 听全文

智元D1开发教程 | Sim2Real全指南（下）——从训练诊断到 Jetson Orin 实机部署

在上篇中，我们已经完成了从策略接入、Lowlevel 控制链打通，到实机首轮部署与部署侧微调的全部关键步骤。到了这里，机器狗已经不再只是“能连上、能跑起来”，而是开始暴露出更真实的问题：速度跟踪是否稳定、原地站立是否干净、过台阶时是否容易打滑、离开桌面级显卡之后又能否真正走向嵌入式平台。

因此，下篇要解决的就是另外两个更接近落地的问题：

第一，**当实机表现不理想时，如何通过训练日志判断接下来应该改哪里**；
第二，**当策略训练完成之后，如何把它从桌面环境迁移到 Jetson Orin 这样的嵌入式设备上，形成真正可移动的实机部署方案。**



色彩争夺战！占领更多地盘
极限枪战大乱斗
进入小游戏

5. 训练参数调整：从“能跑”到“跑得更像样”

当部署端的小范围参数已经调过一轮之后，如果机器人的表现依旧不够理想，那么下一步就不应该继续在真机端反复试错，而是应该重新回到训练侧，结合 TensorBoard 日志去判断：**当前策略到底是没有学会，还是学得还不够好；到底应该改 reward、改命令分布，还是改环境本身。**

这一步的核心，不是“盲调”，而是先学会读图。

5.1 先学会看 TensorBoard

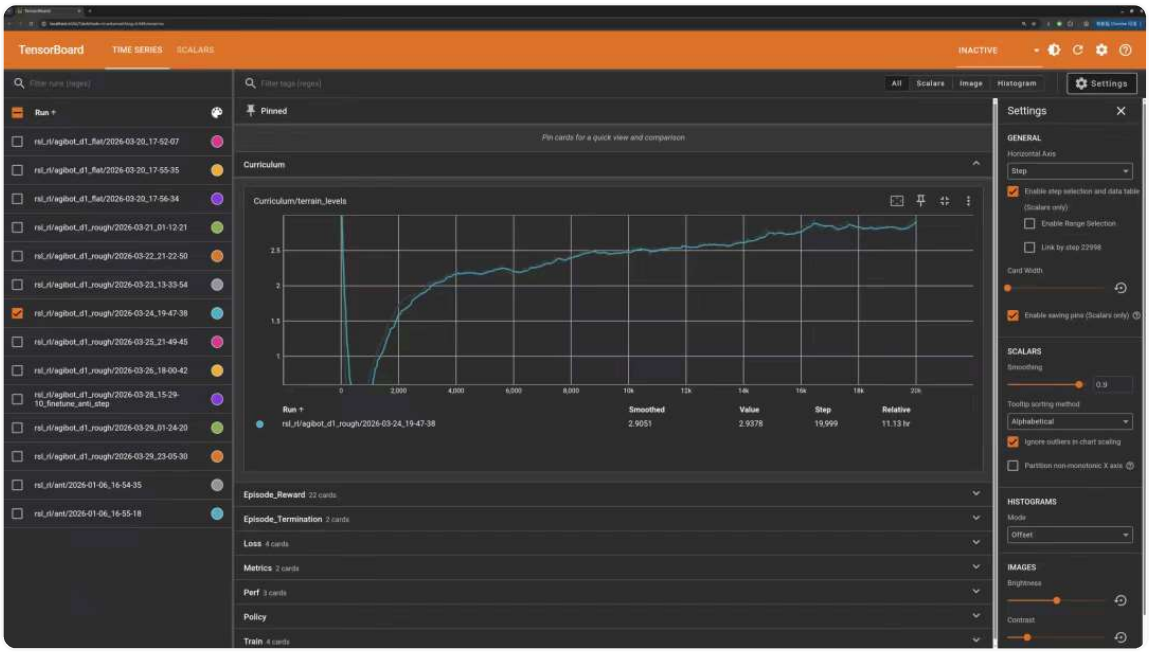
在 `IsaacLab` 文件夹下运行下面的命令：

```
./isaaclab.sh -p -m tensorboard.main --logdir=logs
```

随后根据终端输出访问 `http://localhost:6006/`，就可以打开 TensorBoard 页面。左侧可以选择不同的训练记录，右侧的 **SCALARS** 可以对曲线进行平滑，帮助我们从大量迭代数据中看到真正的趋势，而不是被局部波动带偏判断。

接下来，我们按照几个最关键的分组，逐项分析这组训练结果。

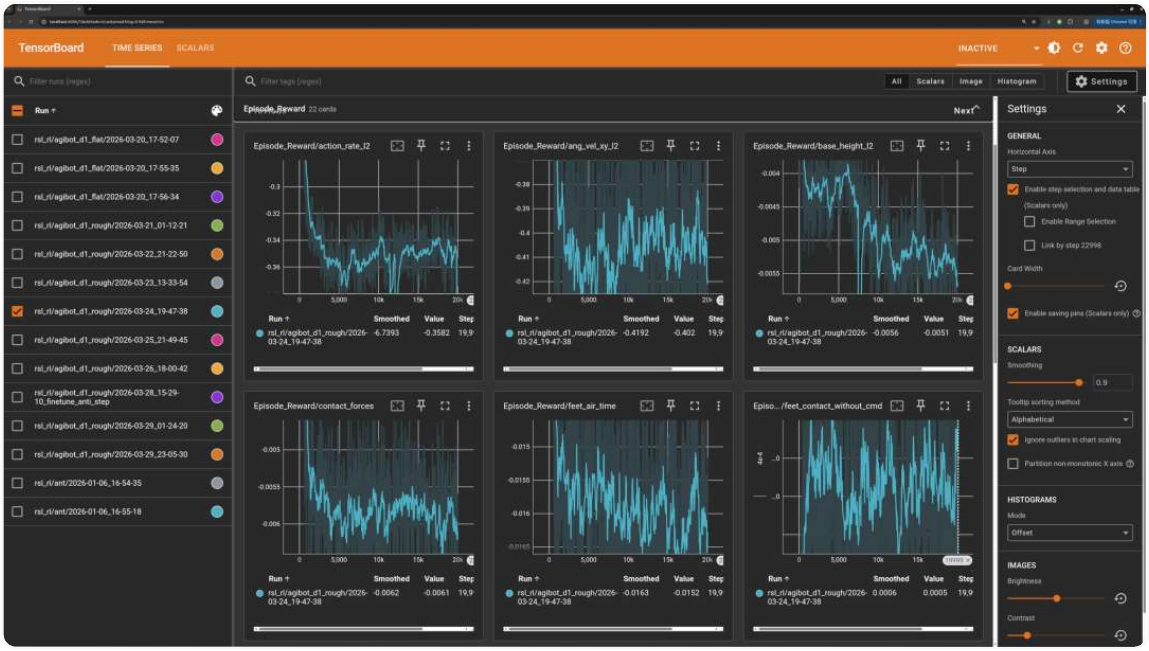
5.1.1 Curriculum：课程学习是否正常推进



先看 `Curriculum/terrain_levels`。这条曲线呈现出“先下降、后上升”的趋势，整体是合理的。它说明训练初期环境给出的地形难度偏高，策略还不足以稳定通过，于是课程学习机制主动把难度下调；而在模型逐渐具备基础通过能力之后，系统又开始稳步提升地形等级。

这类曲线的意义在于：**课程学习确实在发挥作用，而不是始终把机器人困在某一个固定难度里。**
如果一条课程曲线从头到尾都没有明显变化，往往意味着课程策略本身没有生效，或者当前奖励设计无法推动策略跨过难度门槛。

5.1.2 Episode_Reward：奖励项到底在鼓励什么、惩罚什么



这一组里可以先抓几个最有代表性的项来看：

- `action_rate_l2` 依然偏高，说明动作变化较快，控制不够平滑；
- `ang_vel_xy_l2` 在逐步改善，说明横滚与俯仰方向的扰动正在被抑制；
- `contact_forces` 整体比较稳定，说明接触冲击没有出现明显恶化；
- `feet_air_time` 波动较大，意味着步态节奏还不够稳定，抬腿与落脚时序仍在调整。

从这一页可以大致判断：**策略已经开始学会“走”，但走得还不够稳、不够顺。**



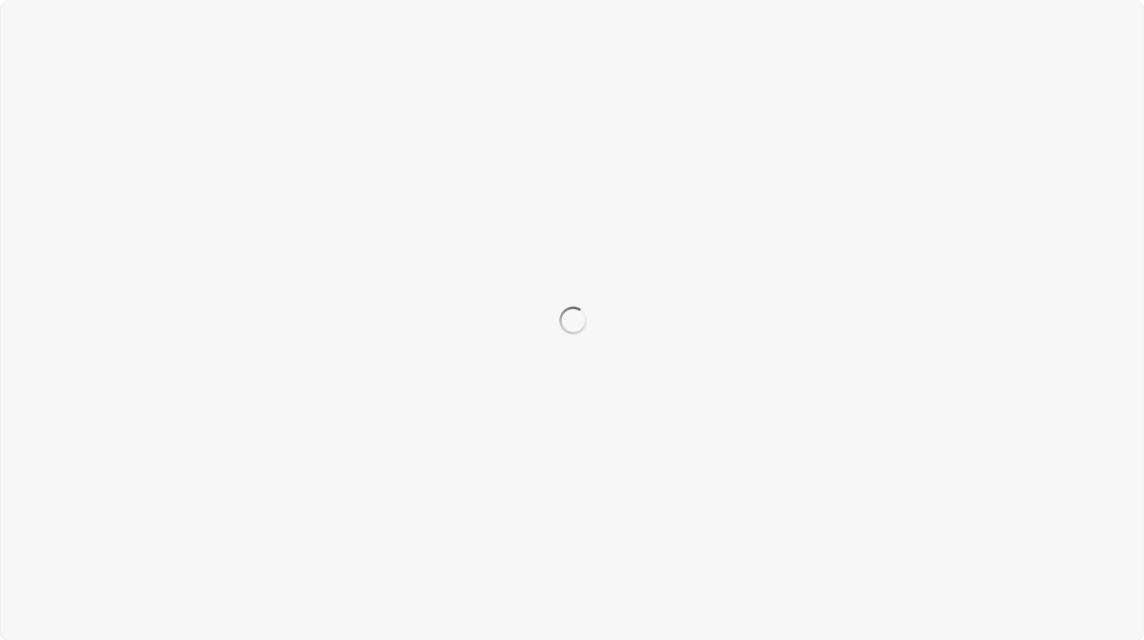
再往后看：

- `feet_height` 说明抬脚高度整体在优化；

- `feet_height_body` 没有形成很稳定的趋势，说明脚相对机身的高度控制还没有完全收敛；
- `feet_slide` 一直存在，说明策略仍有打滑问题；
- `feet_stumble` 偶尔出现，说明还有绊脚现象，但暂时不是最主要矛盾；
- `joint_acc_l2` 偏大，说明关节加速度较高，动作风格略“猛”；
- `joint_mirror` 表现较好，说明左右对称性总体正常。

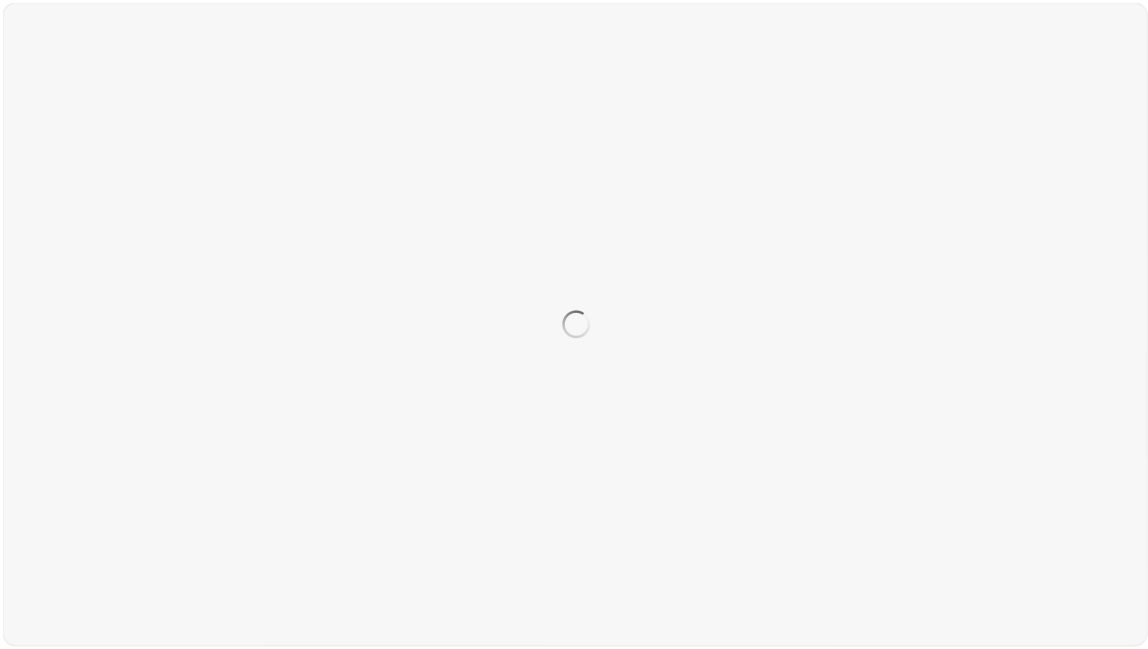
这部分通常对应的是“步态品质”问题。机器人可能已经具备通过能力，但离“干净”“轻盈”“自然”还有距离。

继续点 **Next** 看后续记录：



这一页里：

- `joint_pos_limits` 说明策略偶尔会逼近关节极限，但尚未成为主问题；
- `joint_pos_penalty` 仍然较大，说明关节姿态经常偏离理想参考位；
- `joint_power` 持续下降，这是一个正向信号，说明整体能耗在改善；
- `joint_torques_l2` 偏高，说明虽然能耗下降了，但力矩使用仍不够克制；
- `lin_vel_z_l2` 在下降，说明竖直方向抖动在减小；
- `stand_still` 波动依然明显，说明零命令下的静止质量一般。

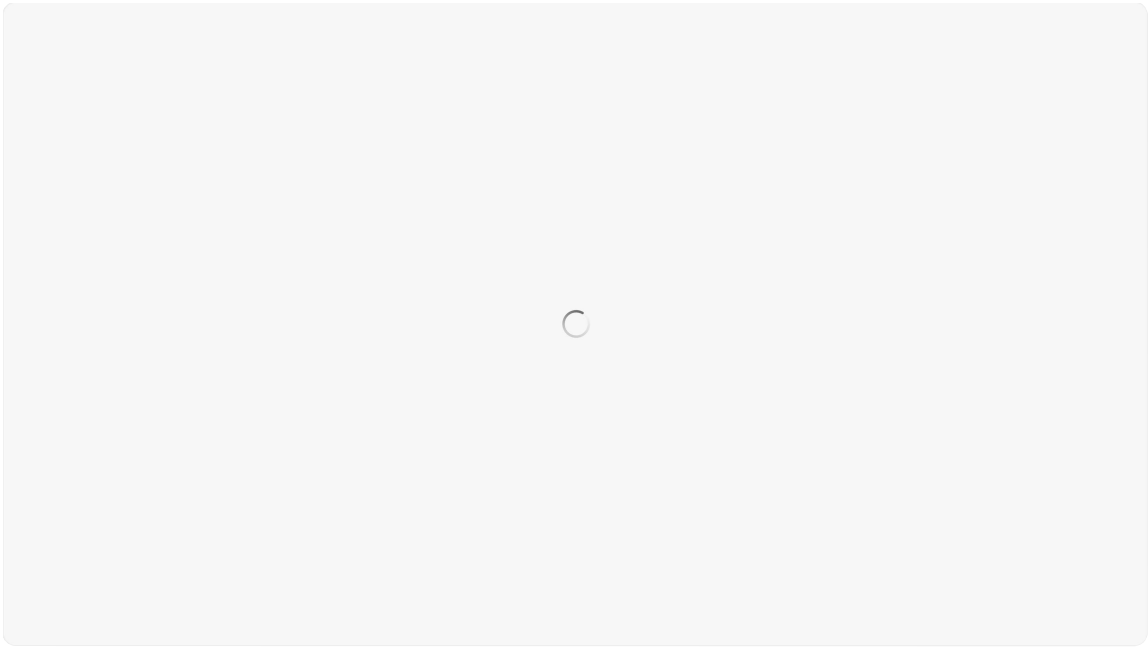


最后一页里：

- `track_ang_vel_z_exp` 说明角速度跟踪在变好，但还不算特别稳；
- `track_lin_vel_xy_exp` 提升更明显，说明速度跟踪能力确实在持续增强；
- `undesired_contacts` 依然存在，说明还有错误接触；
- `upward` 长期稳定，说明机器人总体可以保持直立姿态。

这一组奖励项的核心作用，不只是看“奖励总和涨没涨”，而是拆开看：**策略到底是在哪些能力上在变强，又在哪些地方还留有明显短板。**

5.1.3 Episode Termination：机器人是怎么“结束”的



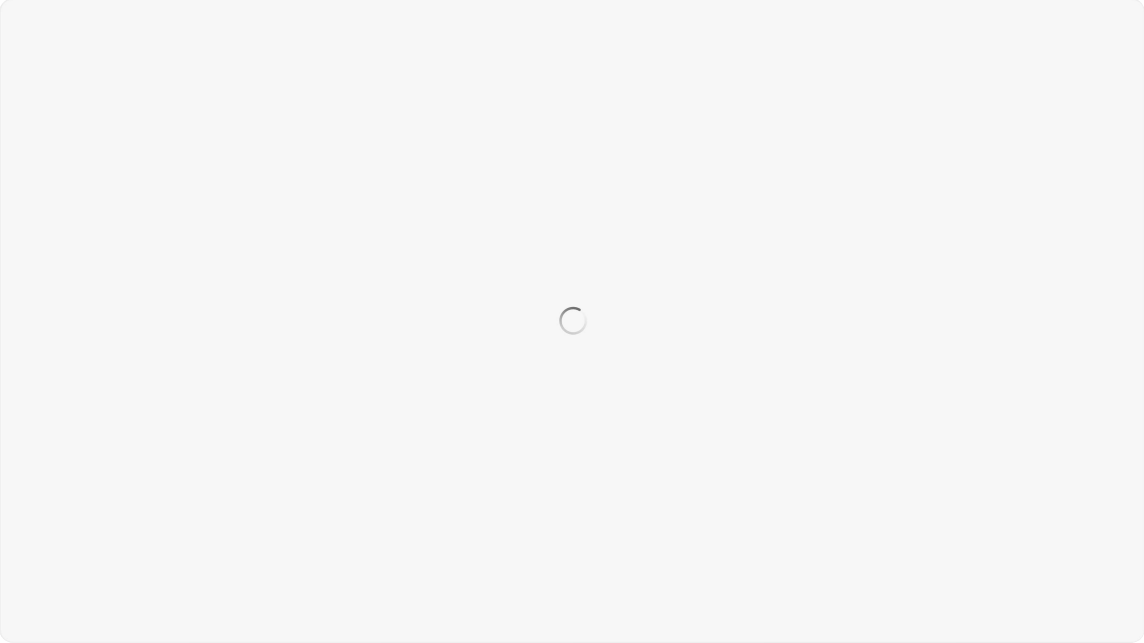
终止原因通常非常重要，因为它直接告诉我们训练失败主要发生在哪种情形。

这组图里只有两项比较突出：

- `time_out` 占大多数，说明绝大部分 episode 都能正常跑满；
- `terrain_out_of_bounds` 很低，说明机器人基本没有大面积跑飞或冲出场景边界。

这个结果说明当前策略至少已经没有严重失控问题。也就是说，它现在面临的更像是“品质优化”问题，而不是“根本不会走”。

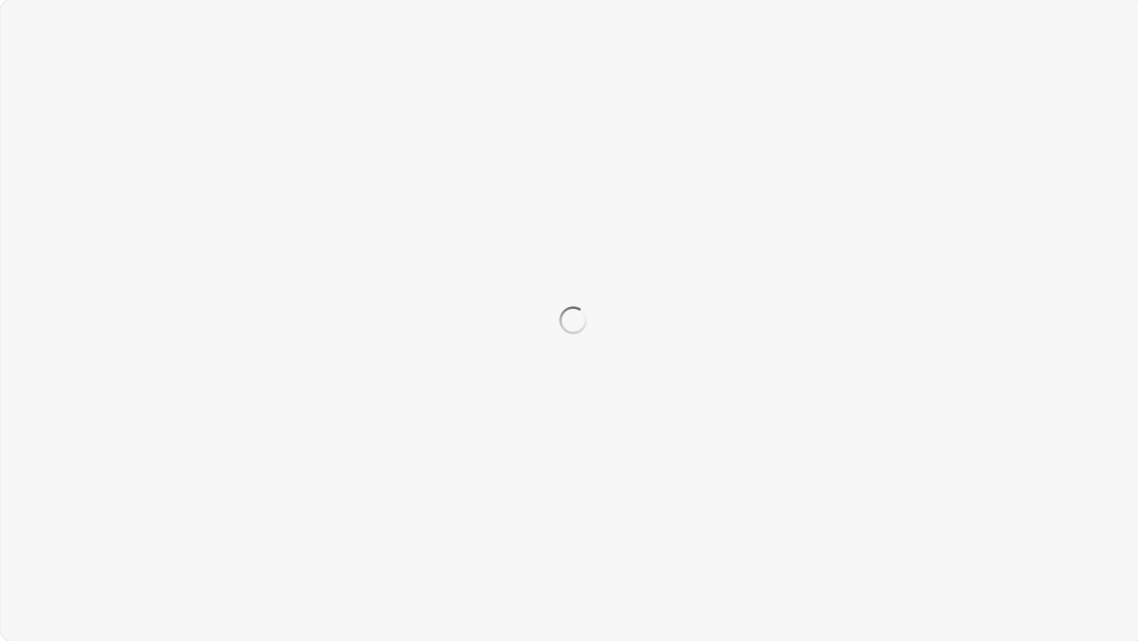
5.1.4 Loss：损失曲线在告诉我们什么



- `entropy` 持续下降，说明探索在减少，策略逐渐进入收敛阶段；
- `surrogate` 整体比较平稳，没有明显发散迹象；
- `value` 大体稳定，但会出现偶发尖峰，这通常意味着在课程升级或分布变化时，价值网络会短时失配；
- `learning_rate` 正常衰减，没有异常。

其中最值得注意的是 `value` 的尖峰。它不一定代表训练失败，但通常意味着：**环境难度、命令分布或奖励结构发生变化时，价值函数需要重新适应。**
如果尖峰只是偶发并且之后能回落，一般属于正常现象；如果频繁出现且越来越高，就需要警惕训练不稳定。

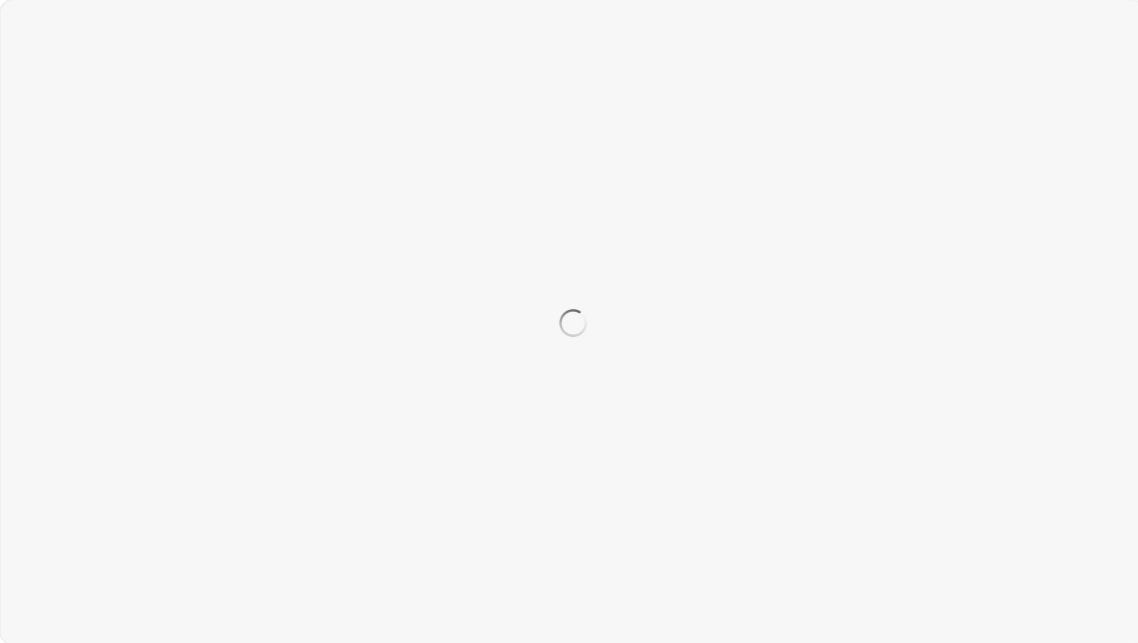
5.1.5 Metrics：真实误差是否在下降



- `error_vel_xy` 在下降，但仍偏大，说明直线速度跟踪还没有完全学稳；
- `error_vel_yaw` 依旧偏高，说明转向控制明显弱于前进控制。

这类图表的好处在于，它给出的不是“代理信号”，而是更直观的真实误差。
如果奖励涨了，但误差始终不降，那么就需要怀疑 reward 设计有没有偏离真正任务目标。

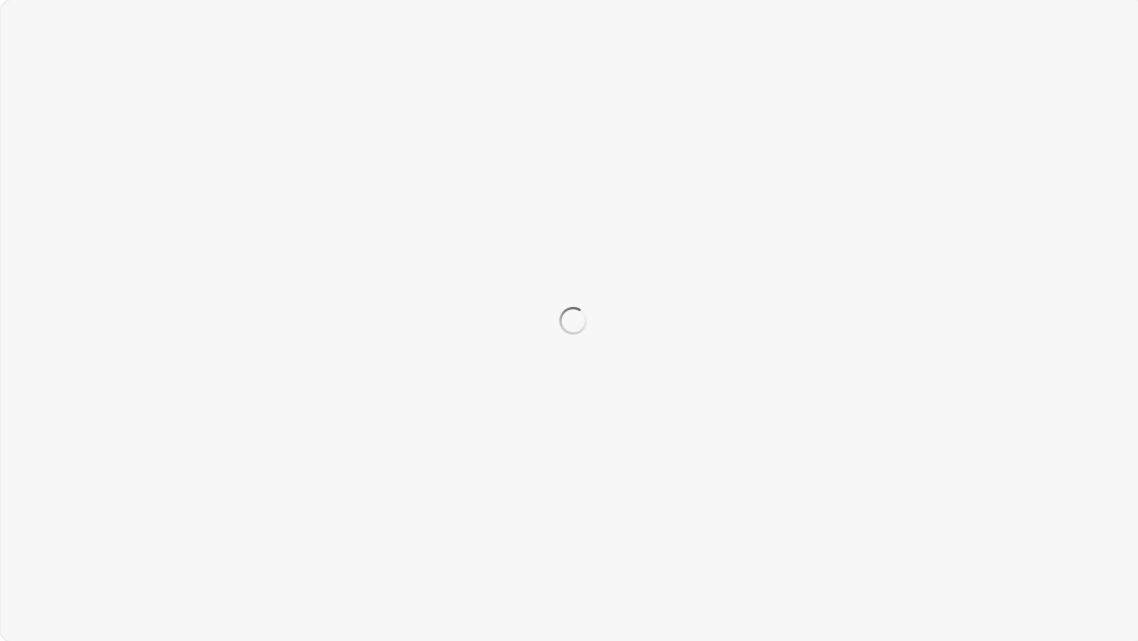
5.1.6 Perf：训练效率是否正常



- `collection_time` 稳定，说明采样侧没有明显瓶颈；
- `learning_time` 平稳，说明网络规模与当前硬件较为匹配；
- `total_fps` 维持在较高水平，说明整体训练效率正常。

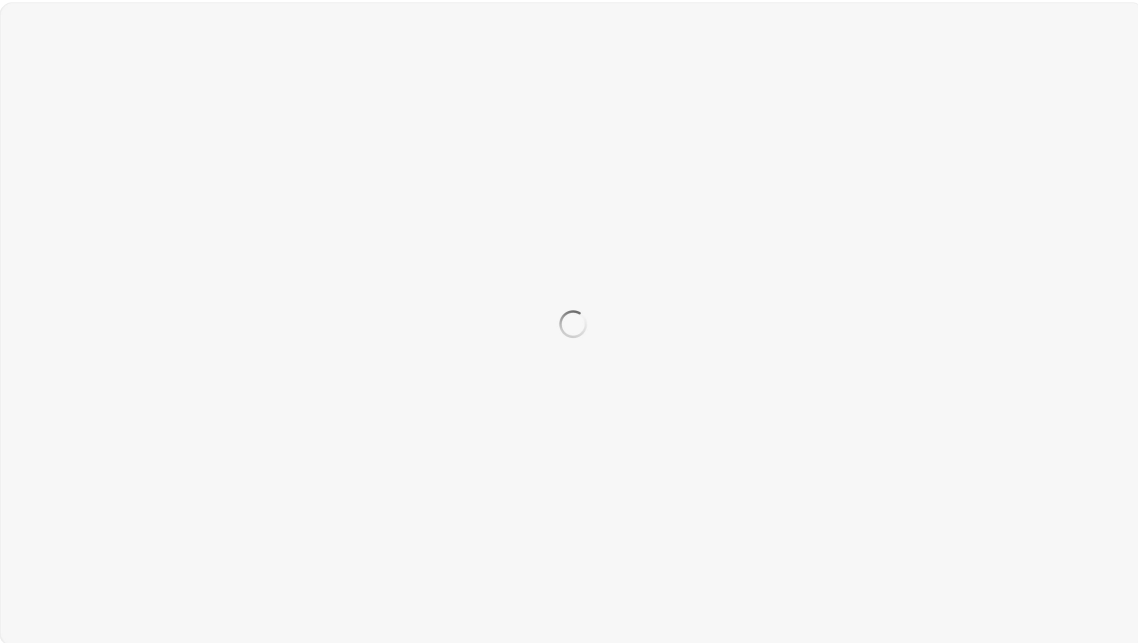
性能曲线的意义在于确认：**当前问题到底来自算法，还是来自系统。**
如果训练质量差，但性能曲线稳定，那么就应优先从环境与奖励侧排查；如果性能本身剧烈波动，则需要先检查仿真、数据采样或硬件资源分配。

5.1.7 Policy：策略还有多“随机”



`mean_std` 仍有波动，说明策略虽然在收敛，但还没有完全定型。
这类图在中后期尤其有用：如果它始终维持在很高水平，说明策略还在强探索阶段；如果下降过快，则可能意味着探索不足、过早收敛。

5.1.8 Train：总体训练效果



- `mean_episode_length` 长期接近跑满，说明训练过程中没有频繁崩掉；
- `mean_reward` 持续上升，说明策略整体在进步；

- `mean_reward/time` 也同步上升，说明训练效率和策略质量都在改进。

到这里，我们就可以形成一个比较清晰的结论：
这组策略不是“没学会”，而是已经具备基本能力，但还需要在稳定性、停稳能力、转向精度和足端行为上继续优化。

5.2 根据日志回头改训练配置

读完 TensorBoard 之后，真正有价值的事情，是把这些判断重新落回配置文件，而不是停留在“看懂了曲线”。

5.2.1 增加更有针对性的地形样本

```
self.scene.terrain.terrain_generator.sub_terrains["pyramid_stairs"].proportion = 0.2
self.scene.terrain.terrain_generator.sub_terrains["pyramid_stairs"].step_height_rang
self.scene.terrain.terrain_generator.sub_terrains["pyramid_stairs_inv"].proportion =
```

这里额外提高了 `pyramid_stairs` 和 `pyramid_stairs_inv` 的占比，并把台阶高度限制到 0.13 ~ 0.15m。这样做的目的，是让训练数据里更频繁地出现“可上下、可跨越、但又不是特别极端”的台阶样本。
对 D1 这种需要兼顾稳定性与通过性的任务来说，这类样本往往比随机生成的大杂烩地形更有价值。

5.2.2 收紧动作裁剪范围

```
self.actions.joint_pos.clip = {".*": (-3.0, 3.0)}
```

这里把 `joint_pos.clip` 从原先非常宽的 `(-100, 100)` 收紧到 `(-3, 3)`。原因很简单：仿真里允许一个几乎不受约束的动作空间，并不代表真实机器人也应该承担这种输出。
在训练阶段就适当收窄动作边界，有助于减少过激动作，让落脚更干净，也能让后续部署更接近真实可执行范围。

5.2.3 重新分配机身稳定性偏好

```
self.rewards.lin_vel_z_l2.weight = -1.0
self.rewards.ang_vel_xy_l2.weight = -0.16
self.rewards.base_height_l2.weight = -0.8
self.rewards.base_height_l2.params["target_height"] = 0.40
```

这里实际上是在重写“机身稳定”的定义。

- `lin_vel_z_l2` 的惩罚减弱，表示允许机器人在上下方向保留一定弹性；
- `ang_vel_xy_l2` 的惩罚增强，表示更强调机身别乱滚、别大幅俯仰；
- `base_height_l2` 从关闭改为启用，并把目标高度固定在 0.40m，表示希望机身在跨障时不要明显塌腰。

这套修改背后的思路是：
上台阶时，真正重要的不是“身体一点都不动”，而是“身体总体稳、姿态别乱，同时腿还能有足够动作空间”。

5.2.4 单独强化“停住不动”的能力

```
self.rewards.stand_still.weight = -1.7
self.rewards.stand_still.params["command_threshold"] = 0.06
self.rewards.track_lin_vel_xy_exp.weight = 2.8
self.rewards.track_ang_vel_z_exp.weight = 1.6
self.rewards.zero_cmd_lin_vel_xy_l2 = RewTerm(
    func=mdp.zero_cmd_lin_vel_xy_l2,
    weight=-0.8,
    params={"command_name": "base_velocity", "command_threshold": 0.06},
)
self.rewards.zero_cmd_ang_vel_z_l2 = RewTerm(
    func=mdp.zero_cmd_ang_vel_z_l2,
    weight=-0.45,
    params={"command_name": "base_velocity", "command_threshold": 0.06},
)
```

原版里“站住”主要依赖 `stand_still` 一个项。现在除了把它的触发阈值收紧到 0.06 之外，又额外加入了零命令下的线速度和角速度惩罚项。这样做的目的，是把“站住不动”从一个模糊目标，变成更明确的约束：

- 零命令下不要小碎步；
- 零命令下不要原地轻微扭动；
- 零命令下机体速度应该真正贴近零。

对于真实机器人而言，这一类 reward 经常非常重要。因为实机体验里最容易暴露问题的，不一定是高速奔跑，而恰恰是“你明明没给命令，它却还在轻微晃动”。

5.2.5 用自适应足端离地替代固定抬腿高度

```
self.rewards.adaptive_feet_clearance_world = RewTerm(
    func=mdp.AdaptiveFeetClearanceReward,
    weight=-0.6,
    params={
        "command_name": "base_velocity",
```

```
        "asset_cfg": SceneEntityCfg("robot", body_names=self.foot_link_name),
        "sensor_cfg": SceneEntityCfg("contact_forces", body_names=self.foot_link_name
        "frame": "world",
        "base_height": 0.09,
        "gain": 0.048,
        "tau": 0.20,
        "tanh_mult": 2.0,
        "stumble_ratio": 3.2,
        "command_threshold": 0.06,
    },
)
self.rewards.adaptive_feet_clearance_body = RewTerm(
    func=mdp.AdaptiveFeetClearanceReward,
    weight=-2.0,
    params={
        "command_name": "base_velocity",
        "asset_cfg": SceneEntityCfg("robot", body_names=self.foot_link_name),
        "sensor_cfg": SceneEntityCfg("contact_forces", body_names=self.foot_link_name
        "frame": "body",
        "base_height": -0.14,
        "gain": 0.018,
        "tau": 0.20,
        "tanh_mult": 2.0,
        "stumble_ratio": 3.2,
        "command_threshold": 0.06,
    },
)
```

这部分修改非常关键。
原先更像是在要求机器人始终按一个固定高度抬脚，而现在改成了基于运动命令与接触状态的**自适应离地控制**。它的好处是：脚不再是“死板地抬到某个高度”，而是会根据当前步态与地形需求动态调整。

换句话说，策略不再只是在“重复一个标准步态”，而是在尝试学会：
什么时候该高抬腿，什么时候该更贴地，什么时候为了避免绊碰需要给脚更多余量。

这对台阶地形尤其重要，因为固定抬腿高度往往很难同时兼顾平地效率和障碍通过性。

5.2.6 重写命令分布，让训练目标更贴近任务本身

```
self.commands.base_velocity.heading_command = False
self.commands.base_velocity.rel_heading_envs = 0.0
self.commands.base_velocity.rel_standing_envs = 0.15
self.commands.base_velocity.ranges.lin_vel_x = (-1.5, 1.5)
self.commands.base_velocity.ranges.lin_vel_y = (-0.8, 0.8)
self.commands.base_velocity.ranges.ang_vel_z = (-2.0, 2.0)
```

这里的改动本质上是在明确：当前训练任务不是“追踪绝对朝向”，而是更直接地追踪三类速度命令：

- 前后速度 `lin_vel_x`
- 左右速度 `lin_vel_y`
- 转向角速度 `ang_vel_z`

同时，把 `rel_standing_envs` 提升到 0.15，表示有 15% 的环境专门练习零命令站立。
这会让策略在“会走”和“能停”之间取得更好的平衡，而不是一味向运动能力倾斜。

5.3 增量式训练结果：不是完美，但方向已经变对

这一版 `rough_env_cfg.py` 的目标非常明确：
让模型学到一种更适合台阶与粗糙地形的步态，具体体现在：

- 机身更稳；
- 零命令时更容易停住；
- 足端离地高度更合理；
- 打滑和误碰撞更少；
- 对线速度与角速度命令的跟踪更直接。

从最后一轮训练结果来看，
`track_ang_vel_z_exp` 和 `track_lin_vel_xy_exp` 的提升都比较明显，说明速度命令跟踪确实增强了；`upward` 长期稳定，也说明机身直立性总体不错，没有出现频繁摔倒。

不过问题也并没有完全解决。
例如 `feet_slide` 和 `feet_stumble` 依然提示存在打滑与绊碰；另外，`stand_still` 虽然整体趋于稳定，但实际数值表现反而有所下降。这通常意味着策略开始更偏向速度方向的执行能力，代价是原地静止时可能仍会出现一定晃动。

这其实是强化学习里很典型的权衡：
当某些能力被强化之后，另一些能力往往会暂时让位。
真正的优化过程，不是一次训练解决全部问题，而是一轮一轮地把策略推向更符合真实需求的方向。

6. Jetson Orin 嵌入式部署：让策略离开工作站

如果始终使用桌面级显卡进行推理，那么策略虽然已经能在实验环境里运行，但它依旧很难真正走向移动场景。一方面，这种部署方式天然受限于场地与供电；另一方面，它也无法对应未来更真实的工程应用形态。

因此，在完成工作站上的训练与验证之后，下一步就是把策略迁移到 **Jetson Orin** 这样的嵌入式平台上，验证整套系统是否具备更高的自主性与可移动性。

6.1 Jetson Orin 环境配置

首先需要给 Orin 安装系统。这里使用 NVIDIA 官方提供的 **SDK Manager** 来完成 Jetson 系列设备的软件管理与系统烧录：

```
https://developer.nvidia.com/sdk-manager
```

安装完成后，可以在命令行中直接启动：

```
sdkmanager
```

如果需要选择历史版本，可以使用：

```
sdkmanager --archived-versions
```

首次使用时需要登录 NVIDIA 账号，国内用户也可以通过微信完成注册与登录。

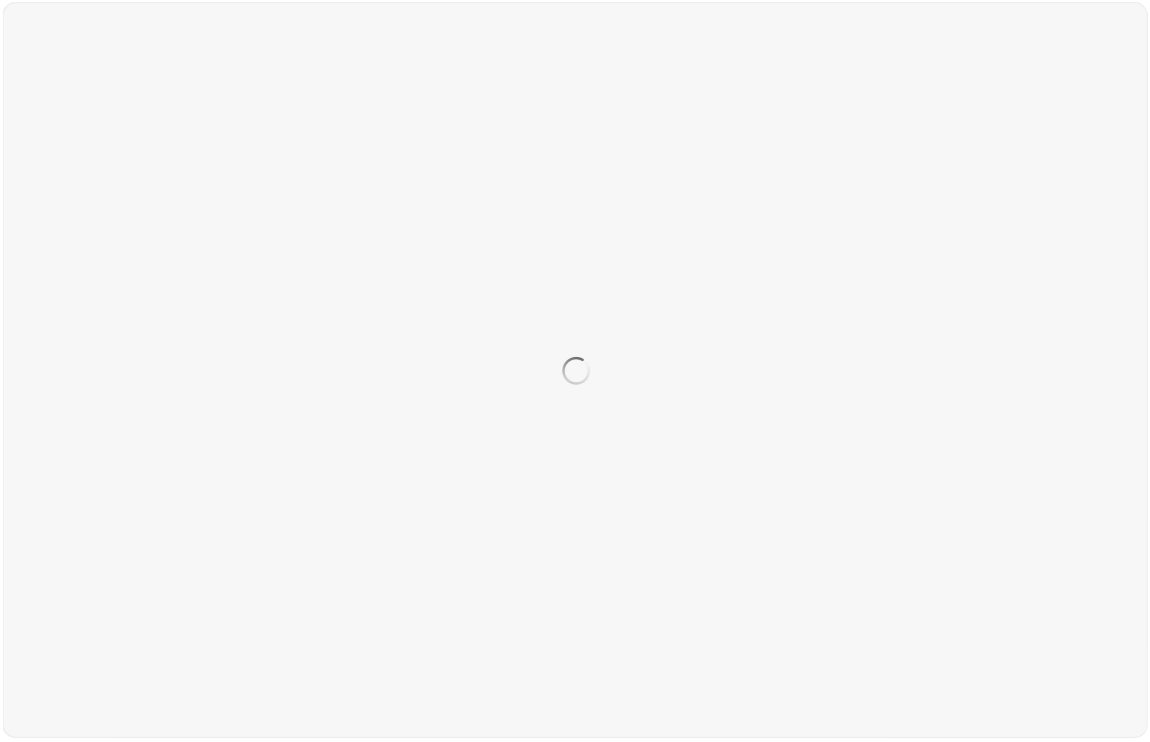
在给 Orin 上电之前，需要先短接 **FC REC** 和 **GND**，让设备进入 Recovery 模式；随后再使用一根 USB 3.0 线连接 Orin 的 Type-C 接口与电脑。

进入 SDK Manager 之后，选择目标设备为 **Jetson Orin NX**，再根据后续推理环境所需的 CUDA、cuDNN 和 TensorRT 版本，选择合适的 JetPack/SDK 版本。

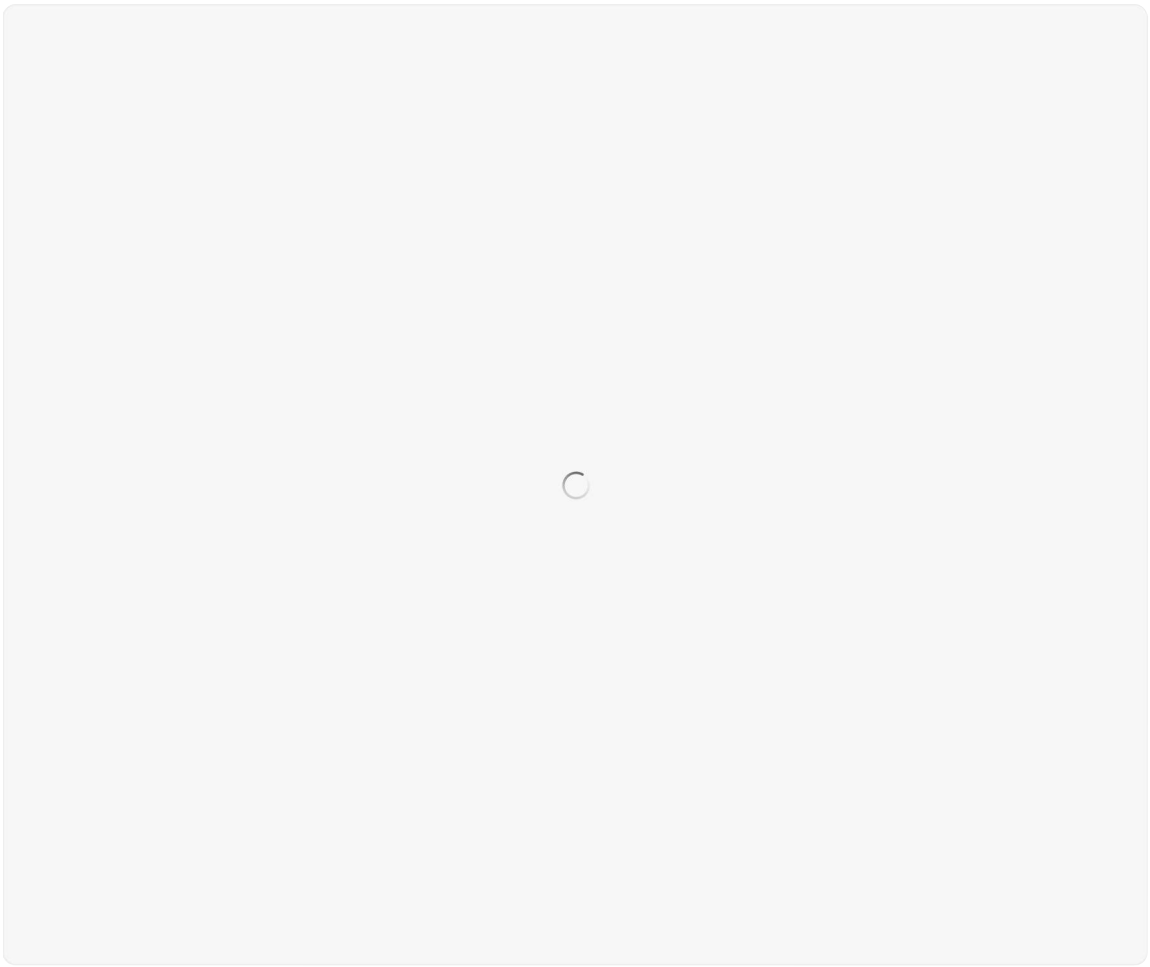
安装组件时，有两点尤其需要注意：

1. **不要勾选更新本机驱动**，否则容易影响当前主机环境；

2. **Runtime 和 SDK 都要安装**，不要只装 Runtime。否则后面如果要进行 CUDA/TensorRT 开发，往往会遇到一系列环境缺失问题。



整个烧录过程中，如果遇到安装失败或者设备无法连接，一个很常见的原因是：电脑同时连接了多个以太网设备，而 Orin 在烧录阶段会把 USB 通道转换成以太网进行数据传输。此时可以优先关闭本机其他以太网，只保留与 Orin 对应的那一路连接。



系统安装完成后，需要将 Orin 的有线网口设置为固定 IP，并放在 `192.168.168.1/24` 网段中。本文的部署示例中，使用的地址是：

```
192.168.168.169
```

6.2 D1 有线连接方式配置

接下来，需要把 D1 切换到有线调试方式。虽然这一部分在前文已经提过，但在嵌入式部署场景下，它是必须重新确认的一步。

首先打开 D1 尾部的有线调试口：取下方形盖板螺丝孔上的硅胶塞，用 2mm 内六角拧下固定螺丝，就可以看到拓展调试接口。

随后使用网线将 D1 调试口与 Orin 的网口直接连接。由于 D1 的调试口本身没有 DHCP 服务器，因此 Orin 端必须手动设置为同一网段。D1 调试口默认地址为：

```
192.168.168.168/24
```

本文示例中，Orin 侧配置为：

```
192.168.168.169/24
```

接着通过 SSH 登录 D1：

```
ssh firefly@192.168.168.168    # 密码为 firefly
```

然后修改 SDK 配置文件，把目标 IP 从本地回环地址改为 Orin 地址：

```
sudo vim /opt/export/config/sdk_config.yaml
```

将其中的 `target_ip` 从 `127.0.0.1` 改成：

```
192.168.168.169
```

接下来继续修改：

```
sudo vim /opt/app_launch/start_motion_control.sh
```

在文件里找到：

```
export SDK_CLIENT_IP="192.168.168.168"
```

这里的地址是 D1 自身的 IP。如果后续机器 IP 发生变化，这一项也必须同步修改。

然后再编辑 systemd 启动文件：

```
sudo vim /etc/systemd/system/robot-launch.service
```

在 `Environment="ROBOT_LOG_DIR=/userdata/log"` 后面增加一行：

```
Environment="ROBOT_NET_INTERFACES=eth0"
```

这一项的作用，是显式告诉系统机器人运控应该绑定哪一个网卡接口。否则在有线调试场景下，很容易出现运控程序绑定失败，最终表现成 SDK 侧无法正常通信。

> *需要特别提醒的是：当配置为有线调试之后，D1 开机时运控程序不会像无线模式那样立即正常启动，而是要求有线调试端口处于载波在线状态。也就是说，调试口必须真的连到另一端网口上，否则运控可能拉不起来。因此这一模式更适合固定部署，而不适合频繁断开测试。*

配置完成后，重启机器，就可以让 Orin 作为 D1 的嵌入式上位机进行控制了。

6.3 硬件连接方式

在硬件上，Orin 的供电范围大约是 5～19V，因此可以直接从机身上的 12V 供电口引出一路电源，通过 XT30 等方式给 Orin 供电；网络侧则通过一根网线与 D1 调试口相连。

理论上，D1 本身是有机械背负接口的，可以进一步设计专用安装支架。不过如果手头暂时没有对应结构件，也可以先采用临时固定方案验证整体链路。本文示例里，就是直接把 Orin 固定在机身上进行测试。

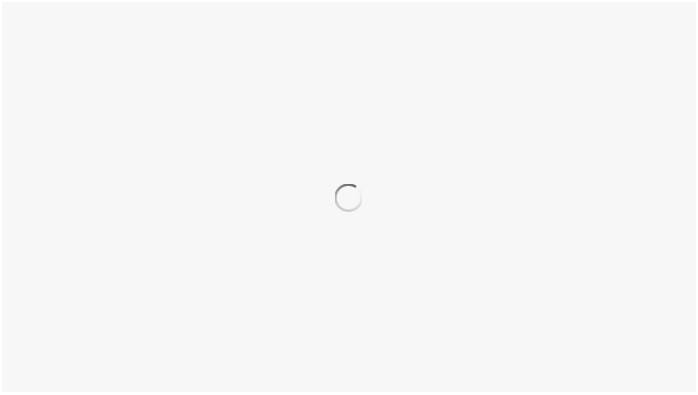
7. 实机运行：让策略真正离开实验台

到了这一步，整套系统已经不再依赖桌面主机，而是由 Orin 直接承担推理与控制任务。
这意味着 Sim2Real 终于不再只是“实验室里的远程控制演示”，而开始接近一种更完整的机载部署形态。

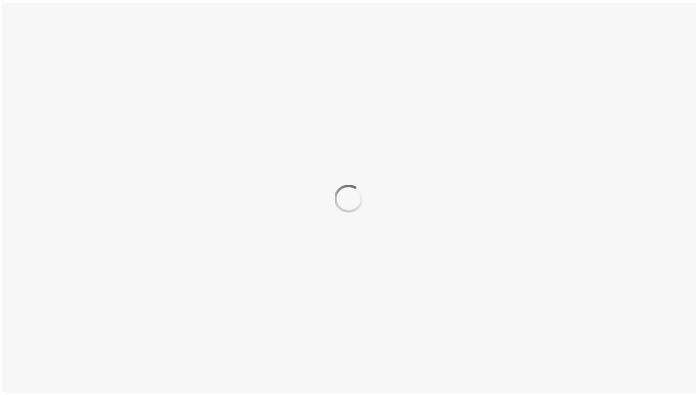
这里还需要补一个工程细节：
由于 D1 本身不能直接把手柄输入上传给 Orin，因此在实机测试中，我们额外修改了 `rl_sar` 中的 `rl_real_d1.py`，让 Orin 外接一个手柄，并由 Orin 读取手柄输入，再把这些输入转换成强化学习策略所需的速度命令与转向命令。这样一来，整条链路就变成了：

外接手柄 → Orin 读取输入 → 生成速度命令 → 策略推理 → Lowlevel 控制下发 → D1 执行动作

从系统结构上看，这一步非常重要。因为它意味着控制输入、策略推理和命令下发已经汇聚在同一台嵌入式设备上，不再需要额外依赖一台桌面工作站作为中介。也正因为如此，后续无论是增加本地视觉感知、加入状态估计模块，还是扩展更复杂的机载算法，系统结构都会更自然。



平地行走



机器越障

从测试结果来看，这套部署方式已经能够完成基础的强化学习控制闭环，说明策略不仅在仿真中成立，也不仅在桌面主机连接实机的方式下成立，而是开始具备真正迁移到嵌入式平台上的可行性。
当然，这并不意味着工作已经结束。恰恰相反，当策略真的被放到机器人背上之后，新的问题才会变得更加具体：

- 嵌入式推理延迟是否足够稳定；
- 手柄输入链路是否会引入额外抖动；
- 机载供电、散热与固定方式是否影响长期运行；
- 当后续再接入视觉或其他传感器时，带宽和算力是否还能支撑。

这些问题都不是训练曲线本身能回答的，而必须在真实部署中一项项被验证。

8. 结语：Sim2Real 不是“跑起来”，而是“跑得住、带得走、用得上”

回过头看，Sim2Real 真正困难的地方，从来都不只是“把一个 `.pt` 模型接到机器人上”。
更难的是在这个过程中，逐步把一套原本只存在于训练环境里的策略，打磨成一个能够承受真实噪声、真实硬件误差、真实控制频率约束的系统。

上篇里，我们解决了控制链路与部署侧问题；
而在下篇里，我们进一步完成了三件更接近落地的事情：

- 学会通过 TensorBoard 判断策略到底差在哪里；
- 学会把这些判断重新落回训练配置本身；
- 学会把策略从桌面主机迁移到 Jetson Orin 这样的嵌入式设备上。



不用你会代码，一键装龙虾。
DClaw-开箱即用的AI助手
查看详情

这三步连在一起，才算是把“训练—部署—优化—再部署”的闭环真正走通。
从这个意义上说，Sim2Real 的结束，并不是某一次实验成功走完了几米路，而是你已经拥有了一套可以持续迭代、持续诊断、持续迁移的完整方法。

而这，往往才是机器人强化学习真正开始变得有工程价值的时刻。

更多ROS、具身智能相关内容，请关注古月居

古月居
专业的ROS机器人知识社区和产业服务平台
1311篇原创内容
公众号

👉 关注我们，发现更多有深度的自动驾驶/具身智能/GitHub 内容！

敢敢AUTOHUB
致力于分享一线具身智能/自动驾驶相关知识，希望能够一起不断学习成长
420篇原创内容

公众号

🚀 往期内容回顾 👀

📘🔥 台北生活 | 101大楼上的谷歌
📘🔥 Jetson Thor 开发指南 | 部署NVIDIA Cosmos Reason 2B视觉语言模型完全教程
📘🔥 智元D1开发教程 | Sim2Real全指南（上）——从控制接入到真机落地

喜欢作者

智元D1开发实战 · 目录 ≡

‹ 上一篇 · 智元D1开发教程 | Sim2Real全指南（上）——从控制接入到真机落地

阅读 193



WorkBuddy AI办公助手，免部署安装即用

WorkBuddy 查看